expert.py

```python
class ExpertSystem:

    def __init__(self):
        self.facts = {}
        self.rules = []
        self.derived_facts = []

    def add_fact(self, fact, value):
        """Add a known fact"""
        self.facts[fact] = value

    def add_rule(self, conditions, conclusion):
        """
        Add a rule to the knowledge base

        conditions: dictionary of {fact: required_value}
        conclusion: string representing the diagnosis
        """
        self.rules.append({
            'conditions': conditions,
            'conclusion': conclusion
        })

    def check_rule(self, rule):
        """
        Check if all conditions in a rule are satisfied.
        Returns True if all conditions match current facts.
        """
        conditions = rule['conditions']

        for fact, required_value in conditions.items():
            if fact not in self.facts or self.facts[fact] != required_value:
                return False

        return True

    def forward_chain(self):
        """
        Execute forward chaining inference.
        """

        changed = True
        iteration = 0

        print("\n" + "=" * 70)
        print("FORWARD CHAINING INFERENCE ENGINE")
        print("=" * 70)

```

```
        print("\nInitial Facts:")

        for fact, value in self.facts.items():
            print(f"  {fact}: {value}")

        while changed:
            changed = False
            iteration += 1

            print(f"\n--- Iteration {iteration} ---")

            for rule in self.rules:

                if self.check_rule(rule):
                    conclusion = rule['conclusion']

                    if conclusion not in self.derived_facts:
                        self.derived_facts.append(conclusion)
                        changed = True

                        print(f"✓ Rule fired: {conclusion}")
                        print(
                            f"  Conditions met: "
                            f"{rule['conditions']}"
                        )

        if not changed:
            print("No new facts derived. Inference complete.")

        return self.derived_facts

    def display_diagnosis(self):
        """Display final diagnosis"""

        print("\n" + "=" * 70)
        print("DIAGNOSIS RESULTS")
        print("=" * 70)

        if self.derived_facts:

            print("\nPossible Diagnoses:")

            for i, diagnosis in enumerate(self.derived_facts, 1):
                print(f"  {i}. {diagnosis}")

        else:
            print(
                "\nNo matching diagnosis found "
                "based on symptoms."
            )
```

```
        print("\n" + "=" * 70)


# ============================================================
# Create Expert System
# ============================================================

print("\n" + "#" * 70)
print("MEDICAL DIAGNOSIS EXPERT SYSTEM")
print("#" * 70)

expert_sys = ExpertSystem()


# ============================================================
# Add Rules
# ============================================================

expert_sys.add_rule(
    {
        'fever': True,
        'cough': True,
        'sore_throat': True
    },
    'Diagnosis: Common Cold'
)

expert_sys.add_rule(
    {
        'fever': True,
        'cough': True,
        'body_ache': True,
        'chills': True
    },
    'Diagnosis: Flu (Influenza)'
)

expert_sys.add_rule(
    {
        'fever': True,
        'cough': True,
        'shortness_of_breath': True,
        'chest_pain': True
    },
    'Diagnosis: Pneumonia'
)

expert_sys.add_rule(
    {
```

```
        'fever': True,
        'cough': True
    },
    'Diagnosis: Possible Respiratory Infection'
)

expert_sys.add_rule(
    {
        'fever': True
    },
    'Recommendation: Monitor temperature closely'
)

expert_sys.add_rule(
    {
        'cough': True
    },
    'Recommendation: Drink warm liquids and rest'
)


# ============================================================
# Add Patient Symptoms
# ============================================================

print("\nPatient Symptoms:")

symptoms = {
    'fever': True,
    'cough': True,
    'sore_throat': True,
    'body_ache': False,
    'chills': False,
    'shortness_of_breath': False,
    'chest_pain': False
}

for symptom, present in symptoms.items():

    expert_sys.add_fact(symptom, present)

    status = "Present" if present else "Absent"

    print(f"  {symptom}: {status}")


# ============================================================
# Run Inference
# ============================================================

```

```
expert_sys.forward_chain()

expert_sys.display_diagnosis()


# ============================================================
# Example 2: Different Patient
# ============================================================

print("\n\n" + "#" * 70)
print("EXAMPLE 2: DIFFERENT PATIENT WITH DIFFERENT SYMPTOMS")
print("#" * 70)

expert_sys2 = ExpertSystem()


# ============================================================
# Add Same Rules
# ============================================================

expert_sys2.add_rule(
    {
        'fever': True,
        'cough': True,
        'sore_throat': True
    },
    'Diagnosis: Common Cold'
)

expert_sys2.add_rule(
    {
        'fever': True,
        'cough': True,
        'body_ache': True,
        'chills': True
    },
    'Diagnosis: Flu (Influenza)'
)

expert_sys2.add_rule(
    {
        'fever': True,
        'cough': True,
        'shortness_of_breath': True,
        'chest_pain': True
    },
    'Diagnosis: Pneumonia'
)

expert_sys2.add_rule(
```

```
    {
        'fever': True,
        'cough': True
    },
    'Diagnosis: Possible Respiratory Infection'
)

expert_sys2.add_rule(
    {
        'fever': True
    },
    'Recommendation: Monitor temperature closely'
)


# ============================================================
# Patient 2 Symptoms
# ============================================================

print("\nPatient 2 Symptoms:")

symptoms2 = {
    'fever': True,
    'cough': True,
    'sore_throat': False,
    'body_ache': True,
    'chills': True,
    'shortness_of_breath': False,
    'chest_pain': False
}

for symptom, present in symptoms2.items():

    expert_sys2.add_fact(symptom, present)

    status = "Present" if present else "Absent"

    print(f"  {symptom}: {status}")


# ============================================================
# Run Inference
# ============================================================

expert_sys2.forward_chain()

expert_sys2.display_diagnosis()
```

```
============================================================
PS C:\Users\Student.DESKTOP-R275QHQ\python folder 1> python -u "c:\Users\Student.DESKTOP-R275QHQ\python folder 1\expert.py"

######################################################################
MEDICAL DIAGNOSIS EXPERT SYSTEM
######################################################################

Patient Symptoms:
  fever: Present
  cough: Present
  sore_throat: Present
  body_ache: Absent
  chills: Absent
  shortness_of_breath: Absent
  chest_pain: Absent

======================================================================
FORWARD CHAINING INFERENCE ENGINE
======================================================================

Initial Facts:
  fever: True
  cough: True
  sore_throat: True
  body_ache: False
  chills: False
  shortness_of_breath: False
  chest_pain: False

--- Iteration 1 ---
✓ Rule fired: Diagnosis: Common Cold
  Conditions met: {'fever': True, 'cough': True, 'sore_throat': True}
✓ Rule fired: Diagnosis: Possible Respiratory Infection
  Conditions met: {'fever': True, 'cough': True}
✓ Rule fired: Recommendation: Monitor temperature closely
  Conditions met: {'fever': True}
✓ Rule fired: Recommendation: Drink warm liquids and rest
  Conditions met: {'cough': True}

--- Iteration 2 ---
No new facts derived. Inference complete.
```

Activate Windo

```
######################################################################
EXAMPLE 2: DIFFERENT PATIENT WITH DIFFERENT SYMPTOMS
######################################################################

Patient 2 Symptoms:
  fever: Present
  cough: Present
  sore_throat: Absent
  body_ache: Present
  chills: Present
  shortness_of_breath: Absent
  chest_pain: Absent

======================================================================
FORWARD CHAINING INFERENCE ENGINE
======================================================================

Initial Facts:
  fever: True
  cough: True
  sore_throat: False
  body_ache: True
  chills: True
  shortness_of_breath: False
  chest_pain: False

--- Iteration 1 ---
✓ Rule fired: Diagnosis: Flu (Influenza)
  Conditions met: {'fever': True, 'cough': True, 'body_ache': True, 'chills': True}
✓ Rule fired: Diagnosis: Possible Respiratory Infection
  Conditions met: {'fever': True, 'cough': True}
✓ Rule fired: Recommendation: Monitor temperature closely
  Conditions met: {'fever': True}

--- Iteration 2 ---
No new facts derived. Inference complete.

======================================================================
DIAGNOSIS RESULTS
======================================================================

Possible Diagnoses:
  1. Diagnosis: Flu (Influenza)
  2. Diagnosis: Possible Respiratory Infection
  3. Recommendation: Monitor temperature closely

======================================================================
```